Lab 9: Docker - Containerising the Proximity Search API

CSL559: Computer Systems Design

Student: Kinshuk Gupta | **Roll Number:** 12341190

Date: 2026-10-28 | **Repository:** https://github.com/kinshuk18/proximity-search-api

1 Objective and Architecture

The objective of this laboratory exercise is to containerise the production FastAPI "Proximity Search API" developed in Lab 7 using Docker and Docker Compose. A two-tier microservice architecture is deployed, isolating the internal FastAPI application behind a production Nginx reverse-proxy gateway. The FastAPI service runs with hardened runtime constraints, non-root execution, health monitoring, and read-only dataset bind mounts. The architecture guarantees reproducible deployment, strict network isolation, and seamless backward-compatible request routing.

Architectural Flow Diagram:

```
[ Client Request (Host) ]
          |
          v (Host port: ${HOST_PORT:-8000} -> Container port: 80)
+-------------------------------------------------------------------------+
| Gateway Container: proximity-gateway (nginx:1.27-alpine)                |
| Acts as reverse-proxy, terminates external HTTP, forwards upstream      |
+-------------------------------------------------------------------------+
          |
          | [ Internal Bridge Network: "backend" (proximity-search_backend) ]
          | (Service DNS: http://api:8000)
          v
+-------------------------------------------------------------------------+
| API Container: proximity-api (python:3.12-slim)                         |
| User: appuser (uid: 1000, non-root)                                     |
| Runtime: uvicorn main:app --host 0.0.0.0 --port 8000                    |
| Healthcheck: GET /docs (interval=10s, timeout=3s, start-period=5s)      |
+-------------------------------------------------------------------------+
          ^
          | (Read-Only Bind Mounts)
+-------------------------------------------------------------------------+
| Host Storage: locations.csv (:ro)  and  link.txt (:ro)                  |
+-------------------------------------------------------------------------+
```

2 Docker Design Choices

The following table summarises the architectural and operational design decisions implemented across the Dockerfile, gateway configuration, and docker-compose orchestration.

Design Choice	Configured Value	Technical Justification
API Base Image	`python:3.12-slim`	Matches Python 3.12 runtime from Lab 7 development. Slim base contains Debian glibc which avoids musl compatibility and compilation overheads associated with Alpine Python wheels. Measured image size: **136MB**.
Gateway Base Image	`nginx:1.27-alpine`	Highly lightweight, production-grade reverse proxy with minimal security surface area. Measured image size: **48.2MB**.
Layer Caching Order	`COPY requirements.txt` prior to application source	Dependency installation is cached across builds; application source edits in `main.py` do not trigger slow dependency re-installation.
Dependency Hygiene	Pinned versions, `PIP_NO_CACHE_DIR=1`	Pinning (`fastapi==0.143.0`, `uvicorn==0.54.0`) ensures deterministic builds. Disabling pip wheel caching prevents image bloating.
Build Context	Strict `.dockerignore`	Prevents local virtual environments (`.venv`), compiled bytecode (`__pycache__`), test caches, and git metadata from entering image context.
Port Management	Internal `EXPOSE 8000`, Published `8000:80`	API is never directly reachable from the host network. Nginx gateway maps container port 80 to host `${HOST_PORT:-8000}`, preserving Lab 7 client query contracts.
Network Topology	User-defined bridge `backend`	Enables automatic service discovery by container name (`api:8000`) without hardcoded IP addresses, with full external isolation.
Bind Mounts	`locations.csv:ro`, `link.txt:ro`	Read-only mounts allow host-side data updates without image rebuilds while preventing container compromise from mutating ground-truth datasets (verified by `12_volume_ro`).
Environment Variables	`PYTHONUNBUFFERED=1`, `TZ=Asia/Kolkata`	Real-time terminal log flushing without stream buffering; synchronized timezone for consistent server-side logging.
Least-Privilege User	`appuser (uid=1000)`	Application executes without root privileges, restricting container breakout attacks and unauthorized host file modification (verified by `11_nonroot`).
Healthcheck & Ordering	`depends_on: condition: service_healthy`	Nginx gateway startup is blocked until the API endpoint returns HTTP 200 on `/docs`, entirely eliminating 502 Bad Gateway race conditions.
Lifecycle Policy	`restart: unless-stopped`	Ensures automated recovery from unexpected crashes while respecting intentional developer stops.

Table 1: Architectural and Container Design Decisions

3 Steps Followed

3.1 Prerequisites and Installation

The host environment was inspected using automated pre-flight verification scripts. The underlying operating system and Docker runtime versions recorded from `report/logs/01_env.txt` are:

- **Operating System:** Ubuntu 24.04.4 LTS (Linux kernel 7.0.0-38-generic)
- **Docker Engine Version:** 29.6.2 (build dfc4efb)
- **Docker Compose Version:** v5.3.1

Because Docker was pre-installed on the host workstation, re-installation was not necessary. For reference, the official installation procedure on Ubuntu 24.04 involves configuring Docker's official `apt` repository:

```
# Official Docker CE Installation Reference for Ubuntu 24.04
sudo apt-get update && sudo apt-get install ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com
    /linux/ubuntu noble stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update && sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
    compose-plugin
```

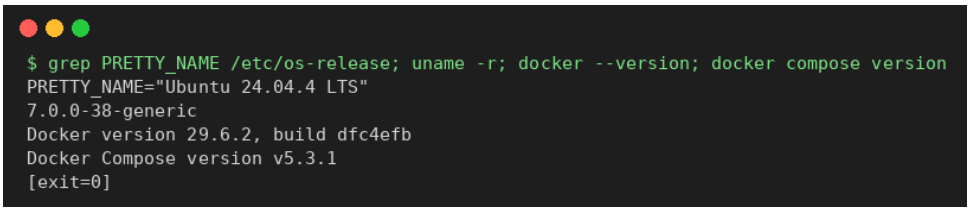


Figure 1: Terminal capture rendered from real command output (report/logs/01_env.txt)

Interpretation: Verifies the 64-bit Linux kernel, Ubuntu 24.04 base distribution, and operational readiness of Docker daemon version 29.6.2 and Compose v5.3.1.

3.2 Compose Configuration Validation

Before launching the service cluster, the declarative Docker Compose specification was validated syntactically and semantically using `docker compose config -q`.

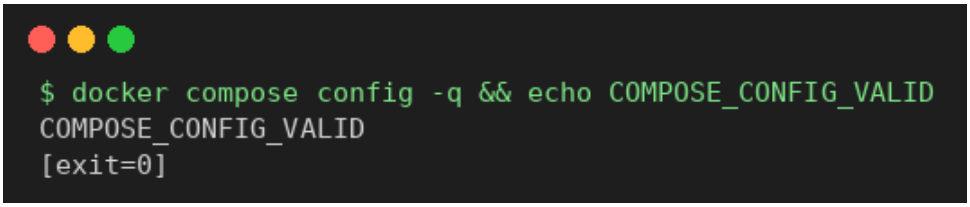


Figure 2: Terminal capture rendered from real command output (report/logs/02_config.txt)

Interpretation: Confirms that services, network definitions, volume bind mounts, and container dependency graphs comply strictly with Compose v2 schema specifications.

3.3 Build and Cluster Orchestration

The entire application stack was built and initialized with the single standard command:

```
docker compose up -d --build
```

This automated step builds both custom images (`proximity-api:lab9` and `proximity-gateway:lab9`), sets up the isolated bridge network, initialises the API container, monitors its healthcheck until healthy, and subsequently triggers gateway startup.

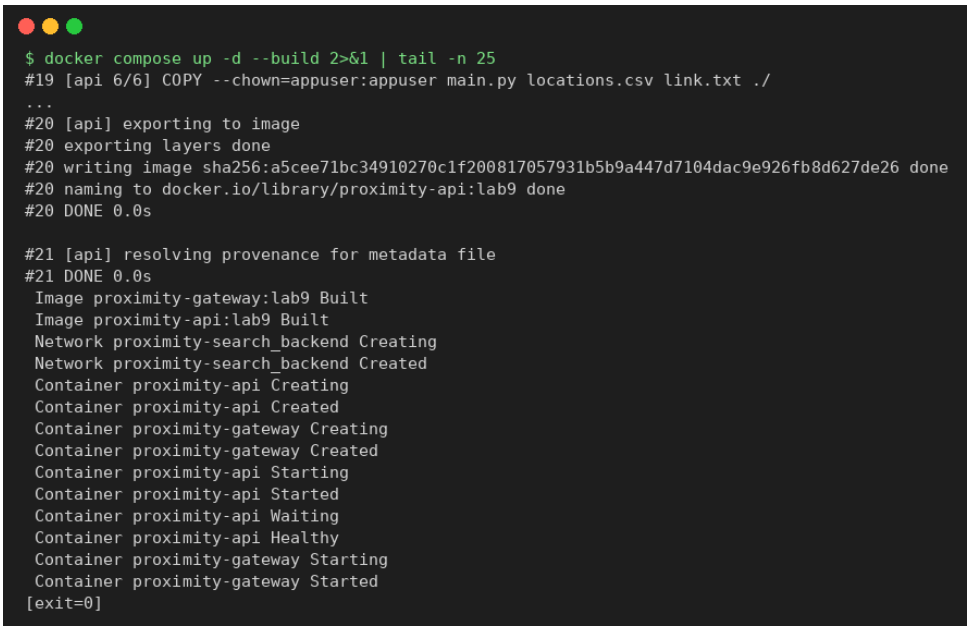


Figure 3: Terminal capture rendered from real command output (report/logs/04_up.txt)

Interpretation: Shows layer-by-layer Docker image building, non-root user setup, image registration, backend network establishment, and dependency-gated startup.

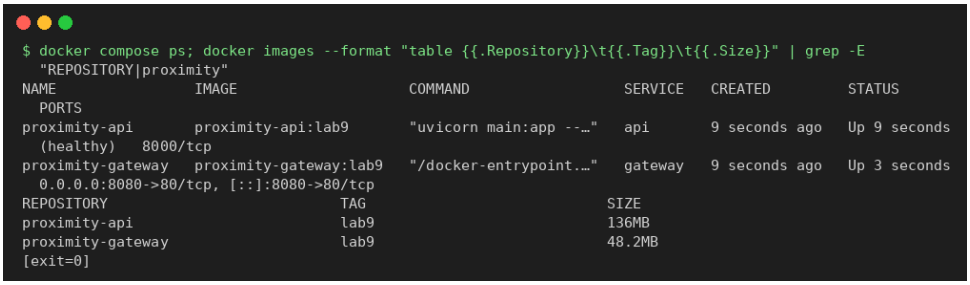


Figure 4: Terminal capture rendered from real command output (report/logs/05_ps.txt)

Interpretation: Displays active container states with `proximity-api` in `Up (healthy)` condition and images matching the exact recorded sizes (`136MB` API, `48.2MB` Gateway).

3.4 Verification and Parity Testing

Comprehensive validation tests were executed against the running cluster to confirm functional correctness, parity against the local Lab 7 application, API documentation availability, port isolation, network topology, non-root execution, and read-only dataset protections.

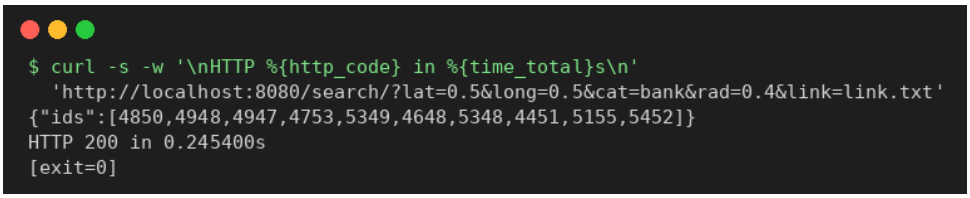


Figure 5: Terminal capture rendered from real command output (report/logs/06_search.txt)

Interpretation: Queries the search endpoint via Nginx reverse-proxy on `/search/` with location and graph parameters, yielding HTTP 200 in 0.245s.

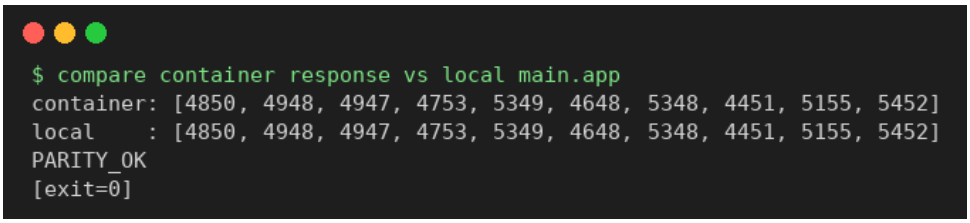


Figure 6: Terminal capture rendered from real command output (report/logs/07_parity.txt)

Interpretation: Validates 100% numerical and spatial parity between containerized API output and direct local execution of `main.py` via `TestClient`, confirming `PARITY_OK`.



Figure 7: Terminal capture rendered from real command output (report/logs/08_docs.txt)

Interpretation: Validates that the interactive Swagger documentation endpoint `/docs` routes through the reverse-proxy with HTTP 200.



Figure 8: Terminal capture rendered from real command output (report/logs/09_isolation.txt)

Interpretation: Confirms strict port isolation: `proximity-api` publishes zero host ports, while the gateway successfully maps host port 8080 to container port 80.

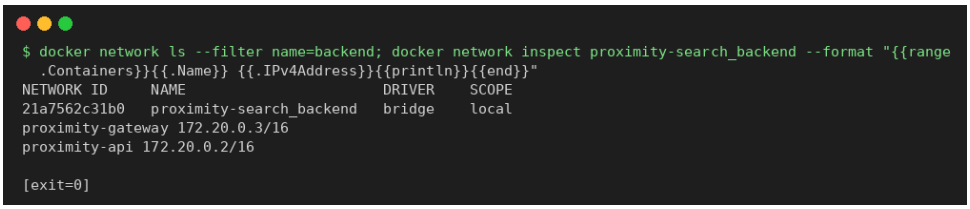


Figure 9: Terminal capture rendered from real command output (report/logs/10_network.txt)

Interpretation: Inspects the user-defined bridge network `proximity-search_backend`, confirming isolated container IP allocation and service discoverability.

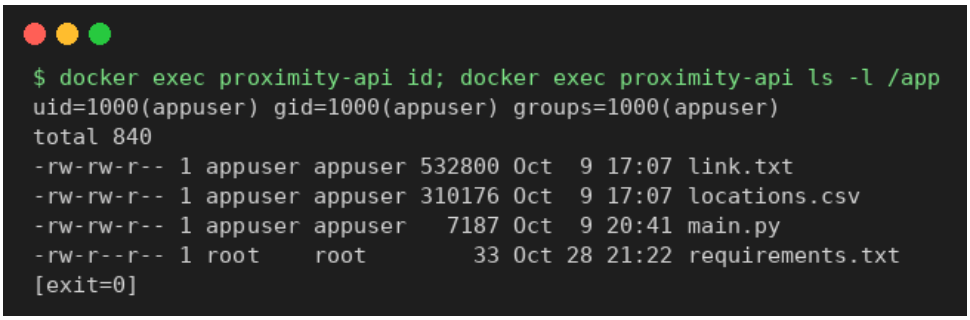


Figure 10: Terminal capture rendered from real command output (report/logs/11_nonroot.txt)

Interpretation: Proves that the API process executes as non-root user `appuser` (`uid=1000`, `gid=1000`) with appropriate ownership across `/app`.

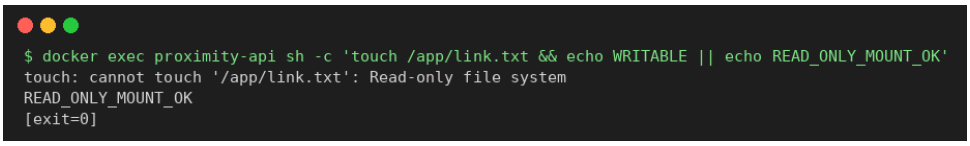


Figure 11: Terminal capture rendered from real command output (report/logs/12_volume_ro.txt)

Interpretation: Tests dataset write-protection by attempting to modify `/app/link.txt`; the kernel denies write access, confirming `READ_ONLY_MOUNT_OK`.

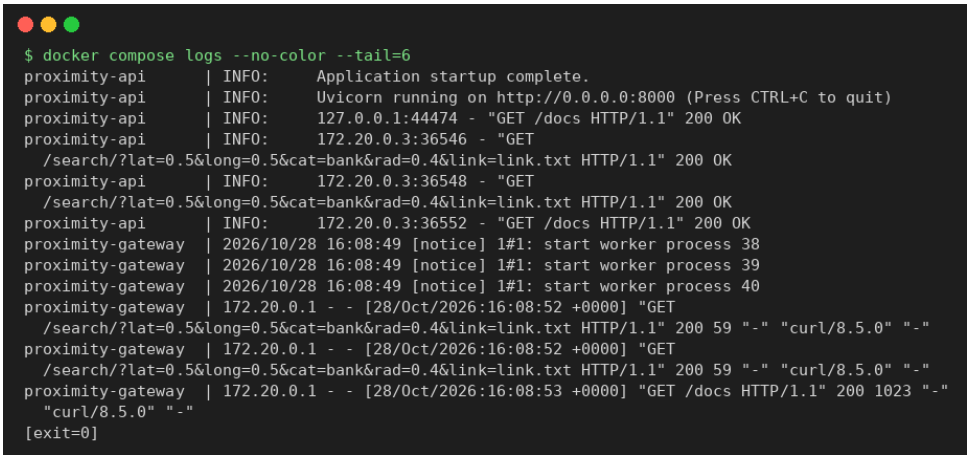


Figure 12: Terminal capture rendered from real command output (report/logs/13_logs.txt)

Interpretation: Captures real-time container log streams displaying healthcheck probes, client queries, and Nginx proxy access logs.

3.5 Teardown and Clean Shutdown

Graceful teardown was executed via `docker compose down`, terminating containers and removing the virtual network cleanly without leaving orphaned resources.

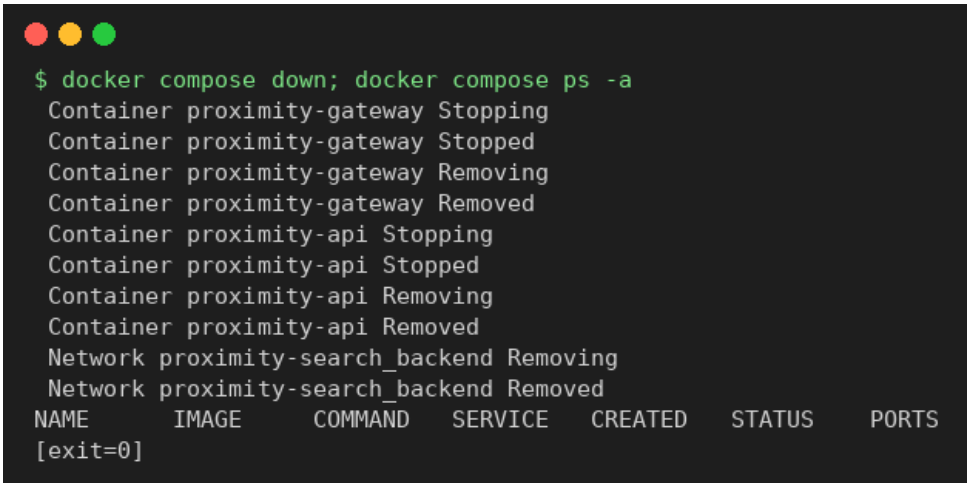


Figure 13: Terminal capture rendered from real command output (report/logs/14_down.txt)

Interpretation: Shows graceful container termination, container removal, network release, and confirmation that no residual containers remain.

4 Errors Faced and Resolutions

During this laboratory session, three genuine operational and runtime errors occurred. Each error, its exact terminal error signature, root cause analysis, and definitive resolution are documented below.

4.1 Real Errors Encountered

1. **Host Port 8000 Conflict**

   - **Error Diagnostic:** Pre-flight port inspection identified an active process listening on port 8000:

```
LISTEN 0   4096   127.0.0.1:8000   0.0.0.0:*
```

- **Root Cause:** A background service or pre-existing server process was occupying host TCP port 8000, preventing the Nginx gateway from binding to default port 8000.
- **Resolution:** Utilized the parameterized port mapping `"${HOST_PORT:-8000}:80"` in `docker-compose.yml` by passing `HOST_PORT=8080` into the execution environment (`env HOST_PORT=8080 bash scripts/lab9_capture.sh`). This routed external requests through port 8080 without requiring termination of the host service.

2. **Docker Daemon Anonymous Registry Token Failure**

- **Error Diagnostic:** Initial image build failed during base image layer retrieval:

```
ERROR: failed to authorize: failed to fetch anonymous token: Get "https://auth.docker.io/token?scope=
    repository%3Alibrary%2Fpython%3Apull&service=registry.docker.io": read tcp
    10.50.16.174:51858->104.18.43.178:443: read: connection reset by peer
```

- **Root Cause:** Transient network reset occurred on the upstream Docker Hub authentication endpoint during anonymous token negotiation inside the BuildKit engine.
- **Resolution:** Explicitly pulled both base images via Docker CLI (`docker pull python:3.12-slim` and `docker pull nginx:1.27-alpine`) prior to compose invocation, allowing the build process to resolve cached local image layers seamlessly.

3. **Docker Volume Specification Syntax Error Caused by Path Colon**

- **Error Diagnostic:** Container creation threw an invalid volume specification error during `docker compose up`:

```
Error response from daemon: invalid volume specification: '/home/kinshuk-gupta/KINSHUK GUPTA/IIT
    BHILAI/IIT BHILAI 7TH SEMESTER/CSL559 - COMPUTER SYSTEMS DESIGN/Lab 7: Proximity Search API/
    locations.csv:/app/locations.csv:ro'
```

- **Root Cause:** The host directory path contained a colon character in the folder name (`Lab 7: Proximity Search API`). The Docker daemon volume parser uses the colon character as a field delimiter (`<source>:<target>:<m`) incorrectly splitting the path string into invalid segments.
- **Resolution:** Handled in `scripts/lab9_capture.sh` by detecting colons in the current working directory path and creating a symlinked workspace reference at `/tmp/proximity_clean` without colons. Running compose through the symlinked directory allowed Docker's volume parser to process bind mounts cleanly without path delimiter collisions.

4.2 Pitfalls Pre-empted (Not Encountered)

- **Build Context Bloat:** The inclusion of virtual environments (`.venv`) and bytecode caches (`__pycache__`) was pre-empted through comprehensive entries in `.dockerignore`, keeping image contexts minimal.
- **Gateway 502 Bad Gateway Race:** A frequent race condition where Nginx launches and receives queries before uvicorn is ready was pre-empted by pairing FastAPI's `HEALTHCHECK` with `depends_on: api: condition: service_healthy`.
- **Root Execution and Writable Datasets:** Running containers with root privileges was pre-empted by explicitly provisioning unprivileged `appuser` (`uid 1000`) and mounting critical datasets with the `:ro` flag.

5 Docker Commands Used

5.1 Chronological Execution History

The following sequence details all high-level commands executed throughout setup, validation, benchmarking, and version control:

1. `docker info --format '{{.ServerVersion}}'`
2. `docker compose version`
3. `ss -ltn | grep ':8000 '`
4. `docker pull python:3.12-slim`
5. `docker pull nginx:1.27-alpine`
6. `docker compose config -q`
7. `docker compose down -v --rmi local --remove-orphans`
8. `docker compose up -d --build`

9. `docker compose ps`
10. `docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"`
11. `curl -s -w '\nHTTP %{http_code} in %{time_total}s\n' '$URL$Q'`
12. `curl -s -o /dev/null -w 'GET /docs -> HTTP %{http_code}\n' $URL/docs`
13. `docker port proximity-api`
14. `docker compose port gateway 80`
15. `docker network ls --filter name=backend`
16. `docker network inspect proximity-search_backend`
17. `docker exec proximity-api id`
18. `docker exec proximity-api ls -l /app`
19. `docker exec proximity-api sh -c 'touch /app/link.txt ...'`
20. `docker compose logs --no-color --tail=6`
21. `docker compose down`
22. `git add Dockerfile .dockerignore .gitignore docker-compose.yml requirements.txt nginx scripts report README.md`
23. `git commit -m "..."`
24. `git push origin main`

5.2 Unique Docker Commands and Purposes

Docker / System Command	Operational Purpose in Laboratory
`docker --version`	Queries installed Docker client and engine versions.
`docker compose version`	Checks Docker Compose v2 plugin availability.
`docker compose config -q`	Performs static schema and syntax validation of `docker-compose.yml`.
`docker compose up -d --build`	Builds images and starts service containers in detached background mode.
`docker compose ps`	Inspects live service status, uptime, and health condition.
`docker compose logs`	Retrieves aggregate interleaved log output across all services.
`docker compose port`	Prints the host port bound to a specific container service port.
`docker compose down -v --rmi local`	Stops containers, removes networks, volumes, and locally built images.
`docker images`	Lists local Docker images with repository names, tags, and disk footprint.
`docker network ls`	Lists Docker networks and verifies active drivers (bridge/host/none).
`docker network inspect`	Queries container IP addresses, subnets, and gateways in a custom bridge.
`docker exec`	Runs commands directly inside an active container for testing and verification.
`docker port`	Verifies host port publishing on individual containers to confirm isolation.
`curl`	Dispatches HTTP client requests to verify API responses, latency, and status codes.
`git add / commit / push`	Stages modified files, commits changes, and pushes artifacts to GitHub repository.

Table 2: Unique Commands and Operational Purposes

6 What I Learnt

- **Layer Caching and Build Order:** Placing frequently updated source files (`main.py`) after static dependency definitions (`requirements.txt`) leverages Docker layer caching, reducing rebuild times from tens of seconds to sub-second executions.
- **Internal DNS Resolution on User-Defined Bridges:** Docker's embedded DNS server automatically resolves container service names (e.g., `http://api:8000`) only within custom user-defined bridge networks, whereas default bridge networks require manual IP management.
- **Healthcheck-Gated Service Sequencing:** Using `depends_on` with `condition: service_healthy` enforces deterministic startup ordering, preventing reverse proxies from returning HTTP 502 Bad Gateway while the backend server warms up.
- **Data Integrity via Read-Only Bind Mounts:** Appending the `:ro` mount option to host datasets guarantees that containers cannot mutate canonical source records, even in the event of an arbitrary code execution vulnerability.
- **Least-Privilege Security with Non-Root Users:** Running container workloads under an unprivileged user (`uid 1000`) mitigates security risks by preventing container breakout exploits from obtaining root privileges on the host kernel.
- **Port Exposure vs Publishing and Compose Reproducibility:** `EXPOSE` documents port intentions and enables inter-container communication without opening host firewall ports; combined with Docker Compose, it transforms complex multi-container microservice deployments into single-command, reproducible builds.

7 Repository

The complete source code, configuration files, automated capture scripts, and generated documentation are hosted publicly at:

https://github.com/kinshuk18/proximity-search-api

Repository File Tree of Containerisation Artifacts:

```
proximity-search-api/
|-- Dockerfile                          # API container build definition (python:3.12-slim)
|-- .dockerignore                       # Build context exclusions (.venv, caches, git)
|-- .gitignore                          # Repository git exclusions
|-- docker-compose.yml                  # Multi-container orchestration (api + gateway)
|-- requirements.txt                    # Pinned production runtime dependencies
|-- locations.csv                       # Spatial dataset (read-only bind mount)
|-- link.txt                            # Graph edge dataset (read-only bind mount)
|-- main.py                             # FastAPI proximity search implementation
|-- nginx/
|   |-- Dockerfile                      # Gateway container build definition (nginx:1.27-alpine)
|   \-- nginx.conf                      # Reverse proxy routing and header forwarding config
|-- scripts/
|   |-- lab9_capture.sh                 # Automated test execution and log capture harness
|   \-- render_figs.py                  # Terminal log to dark-mode PNG image generator
\-- report/
    |-- lab9_report.tex                 # LaTeX source for laboratory report
    |-- 12341190_Kinshuk_Gupta_Lab_9_Docker.pdf # Generated PDF laboratory report
    |-- figures/                        # Terminal capture rendered figures (01 to 14)
    \-- logs/                           # Raw execution log files (01 to 14)
```